

CSE493S Homework #1

William Ong, Noah Shi, Hitender Oswal

May 2, 2026

0 Train Infrastructure Setup

We created training and inference scripts that use the given baseline model code for both language and arithmetic tasks. For modularity, we have two config scripts, model and train config. The model config contains all of parameters for the GPT model while train config handles the training parameters. Our training pipeline allows either a constant learning rate or a cosine decay learning rate as well as the option for a linear warmup. To effectively track training progress, we used TensorBoard for logging. We also save fully contained model checkpoints at every evaluation interval. For both training and inference, we have separate scripts for language and arithmetic. Each script contains logic for correctly encoding the inputs and outputting inference results with the appropriate encoding. The baseline model code did not require any major changes.

0.1 Sanity Checks

For the sanity checks, we used the GPT2 tokenizer to encode the sentence, "I love machine learning." The training and validation data consist of 10 copies of this sentence with proper BOS and EOS tokens around each sentence (60 tokens total). You can find this data under `sanity_check/data` in the submission.

We trained single layer models with 4 attention heads and an embedding size of 256 for this task. You can find the model checkpoint for the regular model in `sanity_check/runs/original` and for the model with the first 3 tokens masked in `sanity_check/runs/masked`. You can run inference with the following command where `path` is the path to the model checkpoint:

```
python inference_language.py --model-path <path> --prompt "<|endoftext|>"  
--temperature 0.5
```

Which will generate the following raw output where "`<|endoftext|>`" is the EOS token:

```
I love machine learning.<|endoftext|>
```

The models were able to successfully memorize the string "I love machine learning." We provide the loss curves below for both models showing convergence:

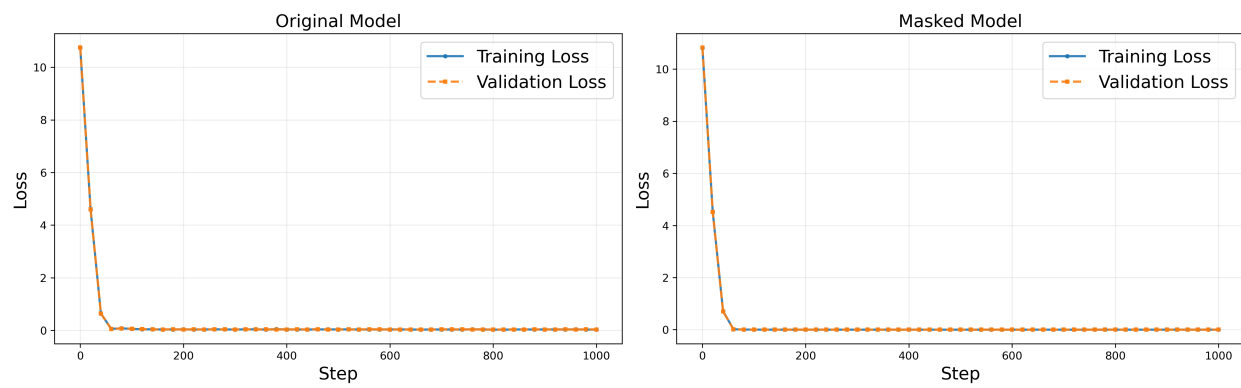


Figure 1: Loss curves for sanity check models

1 Training on Algorithmic Tasks

1.1 Data Generation

We generated data for each possible combination of operand type and p (e.g., addition and $p = 97$, subtraction and $p = 113$, modulo division and $p = 113$). Using a for loop, we streamlined the data generation process where we have a folder for each set of data, labeled $p[p\text{-value}]\text{-}[operand_type]$. Accordingly, each folder contains 3 .npy files for train, test, and validation splits, where we used a 50/40/10 split, as the paper ¹ trained on 50% of the data. This gave us 4704, 3764, and 941 datapoints for the train, validation, and test datasets, respectively. However, the modulo division dataset has slightly fewer datapoints due to the prevention of division by zero, in which those datapoints were discarded. To access the datasets, reference this [link](#) or under `arithmetic_task/data` in the .zip submission.

1.2 Addition and Subtraction Experiments

For the single layer architecture, $p = 97$, and addition operation, we performed 3 runs with random seeds: 0, 29, and 108. Trained on the `p97_add` data, we found the following accuracy/loss curves:

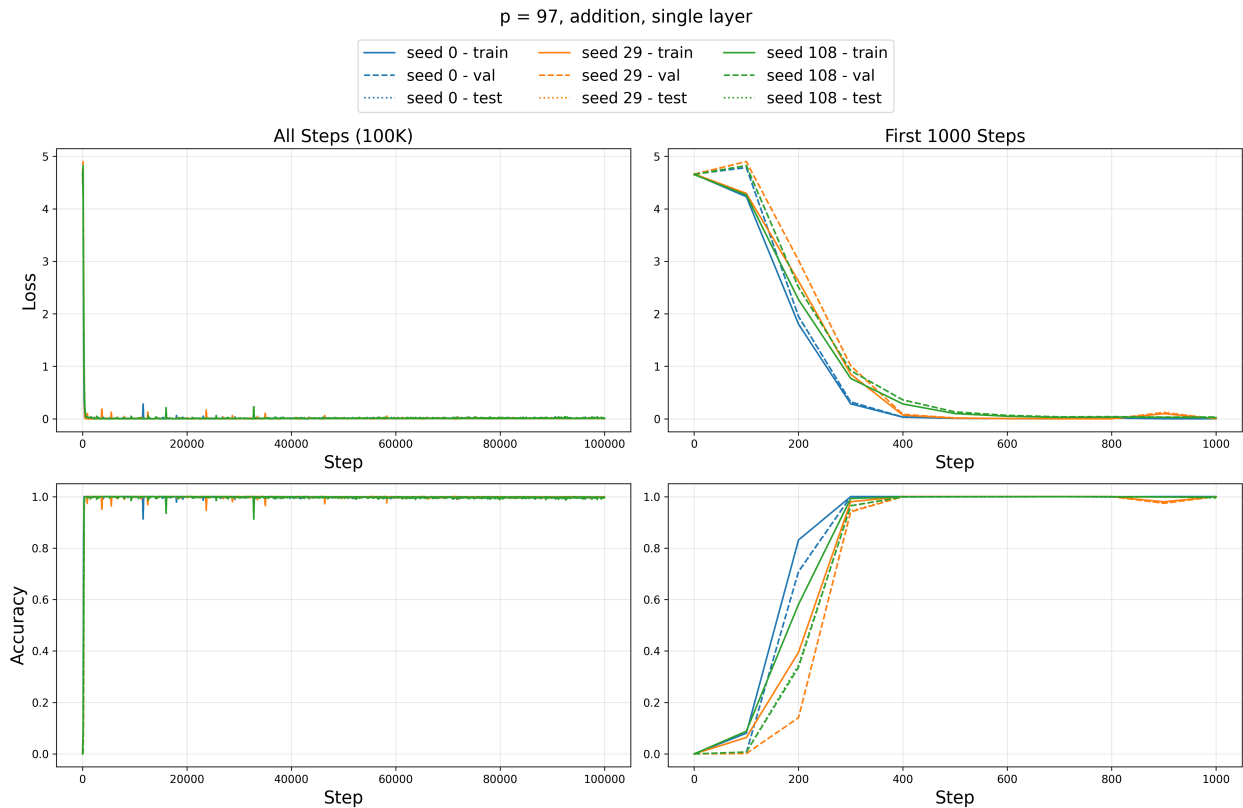


Figure 2: $p = 97$, addition, single layer accuracy/loss curves

¹<https://arxiv.org/abs/2201.02177>

Over the course of 10^5 steps, all seeds reached very close to 100% accuracy and 0 loss across all splits. There were minor differences in the first 1000 steps (as seen in the right visualizations in figure 2), where seed 0 achieved better loss and accuracies earlier than the other seeds. For hyperparameters, we used those similar to the paper ², including AdamW optimizer with a learning rate of 10^{-3} , linear learning rate warmup over the first 10 iterations without any decay, minibatch size of 512, weight decay of 1, $\beta_1 = 0.9$, and $\beta_2 = 0.98$. For checkpoints, reference arithmetic_task/runs in the .zip submission, where each run folder contains its corresponding model checkpoint and TensorBoard events.

We conducted similar experiments for $p = 113$ and subtraction operation, but tested on the two different architectures, 1-layer and 2-layer. Trained on the p113_sub data and analyzing performance across two random seeds 29 and 42, we achieved the following curves:

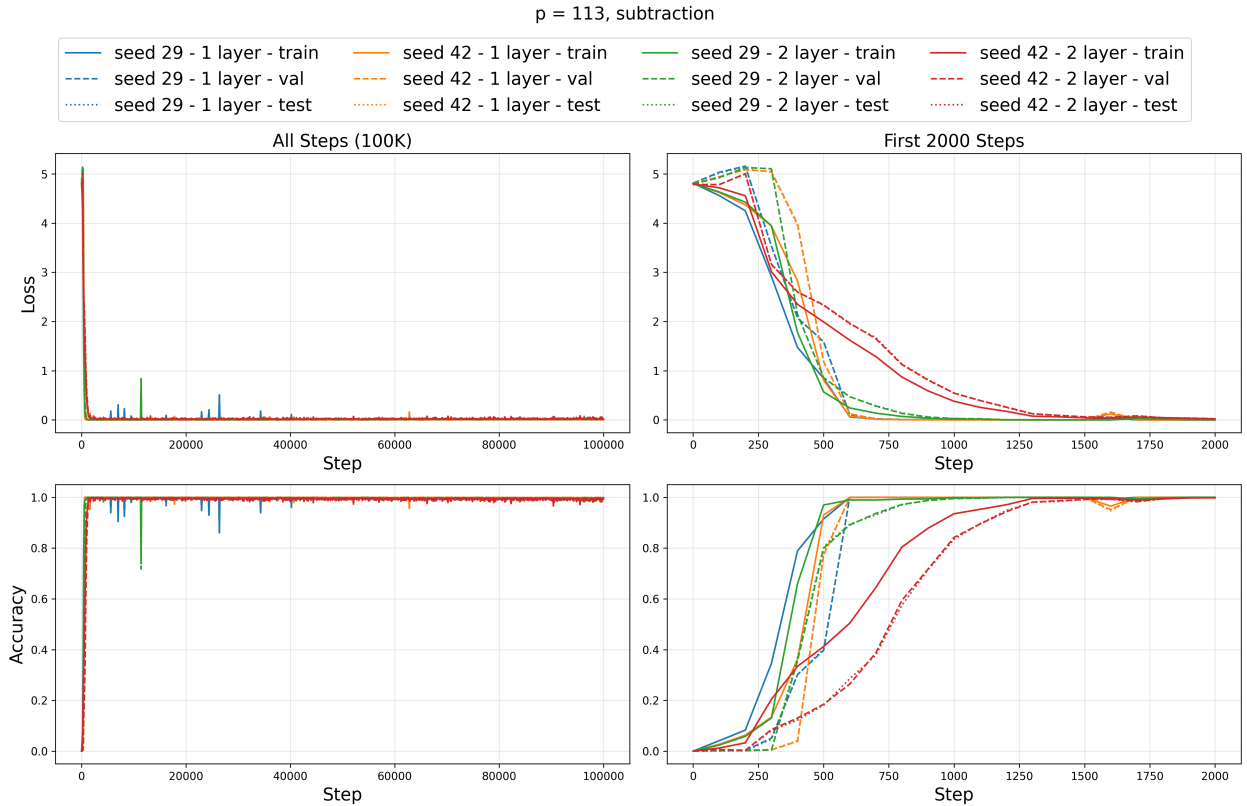


Figure 3: $p = 113$, subtraction accuracy/loss curves

While all architectures and seeds achieved close to 100% accuracy and 0 loss after 10^5 steps, focusing on the first 2000 steps, we can see that the 2-layer architecture on seed 42 took noticeably longer than the other runs to reach the same performance. Still, in the overall context, all runs performed relatively the same across all seeds, splits, and architectures.

For inference, we provide a universal inference script for arithmetic operations, located in arithmetic_task/inference.py. To use it, from the root directory, follow the command below:

²<https://arxiv.org/abs/2201.02177>

```
python arithmetic_task/inference.py --a <1st operand> --b <2nd operand>
--p <modulus> --op <operator> --ckpt-dir <directory of checkpoint>
```

For example, using one of our model's checkpoints and running

```
python arithmetic_task/inference.py --a 14 --b 16 --p 97 --op +
--ckpt-dir arithmetic_task/runs/p97_add_seed0
```

outputs

14 + 16 % 97 = 30

To ensure that inference functions as expected, only use model checkpoints trained on the corresponding p (i.e., if you would like to perform arithmetic operations with $p = 97$, use a $p = 97$ model checkpoint).

1.3 Grokking

To attempt to achieve grokking, we trained on $p = 97$ and division data for the 2 architecture types and seeds, where the loss and accuracy curves can be viewed in figure 4.

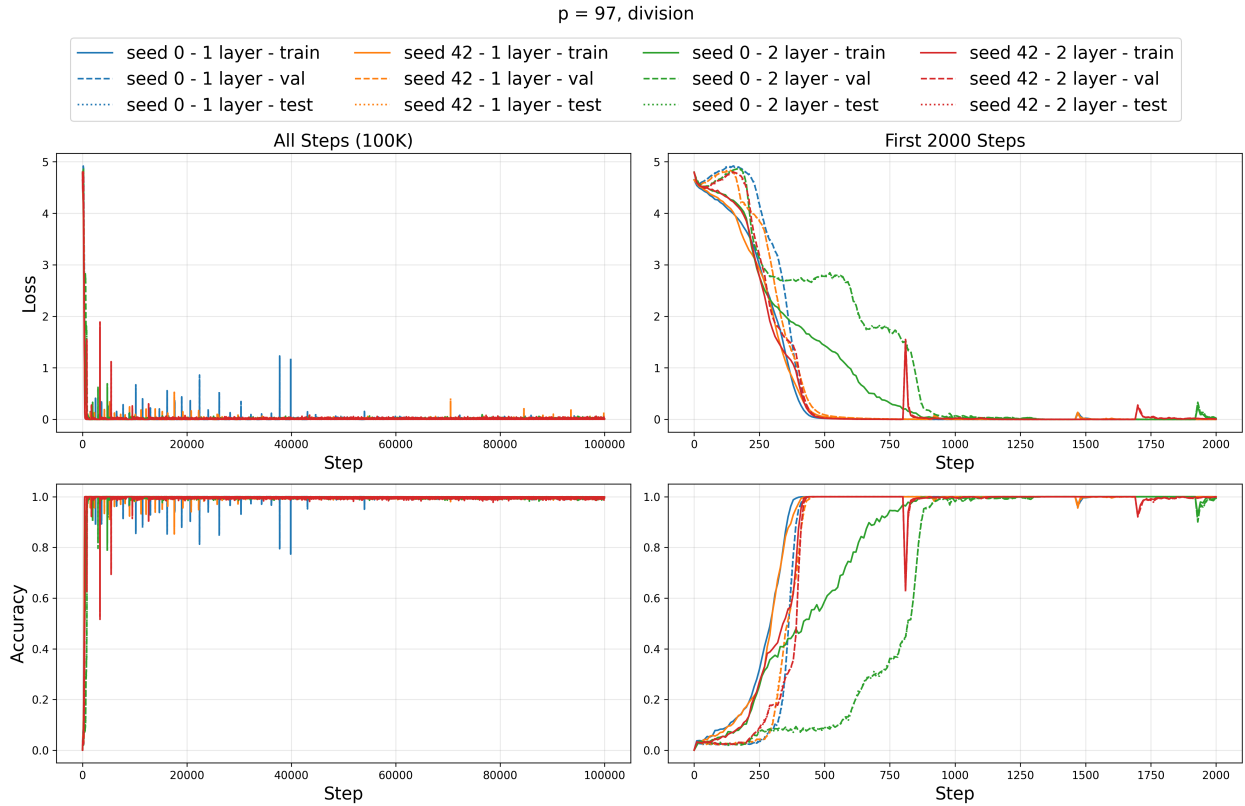


Figure 4: $p = 97$, division accuracy/loss curves

We can see that the train/val/test accuracy curves of 3 of the 4 runs are very closely aligned, indicating that there is no grokking. However, the curves of the 2-layer model on seed 0 (green curves) differs significantly, where there are major discrepancies between the training and test accuracy from steps 500 to 800. Especially from steps 700 to 800, the training accuracy is very close to 100%, yet the test accuracy is significantly lagging behind at around 50%. However, at around step 800, the test accuracy drastically increases, eventually reaching almost 100% accuracy by step 950. This drastic change in test accuracy presents the phenomenon of the model grokking, in which it transitioned from strict memorization to learning the underlying modular division operation. However, this was only a presentation of moderate grokking, as in the larger context (all 10^5 steps), there was no visible grokking, unlike in the referenced paper about grokking. In the next section, we further explore the grokking phenomenon and make the differences much more obvious.

Model checkpoints can be referenced in `arithmetic_task/runs` in the .zip submission. Specifically, the grokking model’s checkpoint is located in `arithmetic_task/runs/p97_div_2layer_seed0/`. Similar to other experiments, for inference, the universal script for arithmetic operations, located in `arithmetic_task/inference.py`, can be used. For example, to perform modular division with $p = 97$, use the following command from the root directory:

```
python arithmetic_task/inference.py --a 50 --b 4 --p 97
    --op / --ckpt-dir arithmetic_task/runs/p97_div_2layer_seed0
```

which outputs

```
50 / 4 % 97 = 61
```

1.4 Analysis

To make the grokking on the division task more noticeable at larger step scales and occur more reliably, we first evaluated the effect of modifying our train/test/val splits. Our hypothesis is that significantly reducing the size of the training dataset will allow the model to memorize the training data quicker, allowing it to reach 100% training accuracy and 0 loss earlier and more reliably, while val/test accuracy would remain low. Since we originally used a 50/40/10 train/val/test split for previous experiments, we updated our split to 25/65/10 for this experiment. Keeping all hyperparameters constant to previous experiments and using a 2-layer model, we achieved the following accuracy and loss curves.

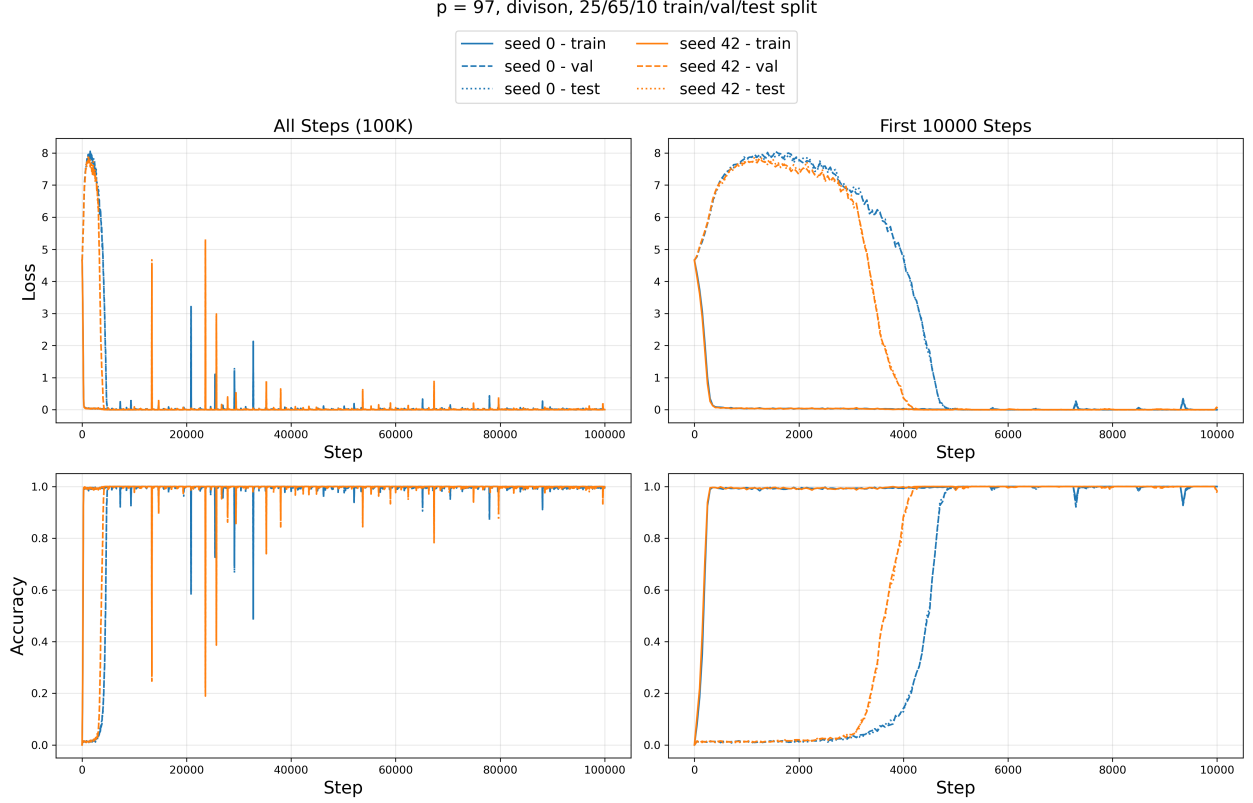


Figure 5: $p = 97$, division accuracy/loss curves, 25/65/10 train/val/test split

We can see that the model memorizes the training data within the first 1000 steps while the val/test loss remain low. It then achieves 100% val/test accuracy around 5000 steps by grokking. To have an accurate comparison to the previous data split, we trained two models with identical seeds as the previous section. Both runs performed similarly and exhibited the same behavior. While this model does not reach 0 val/test loss as quickly as the earlier models, it demonstrates the phenomenon of grokking clearly.

For our second experiment, we adjusted parameters to optimize for the new train/val/test split and stabilize training. We first switched the constant learning rate for a cosine decay learning rate with 10 steps of linear warmup. We kept the learning rate at 10^{-3} while imposing a minimum learning rate of 10^{-5} . Finally, we adjusted the weight decay from 1.0 to 0.6. All other parameters stayed the same and we used a 2-layer model once again. Our hypothesis was that adjusting the parameters in this way would discourage overfitting and lead to a smoother loss curve which could improve training stability. With these parameters, we achieved the following accuracy and loss curves.

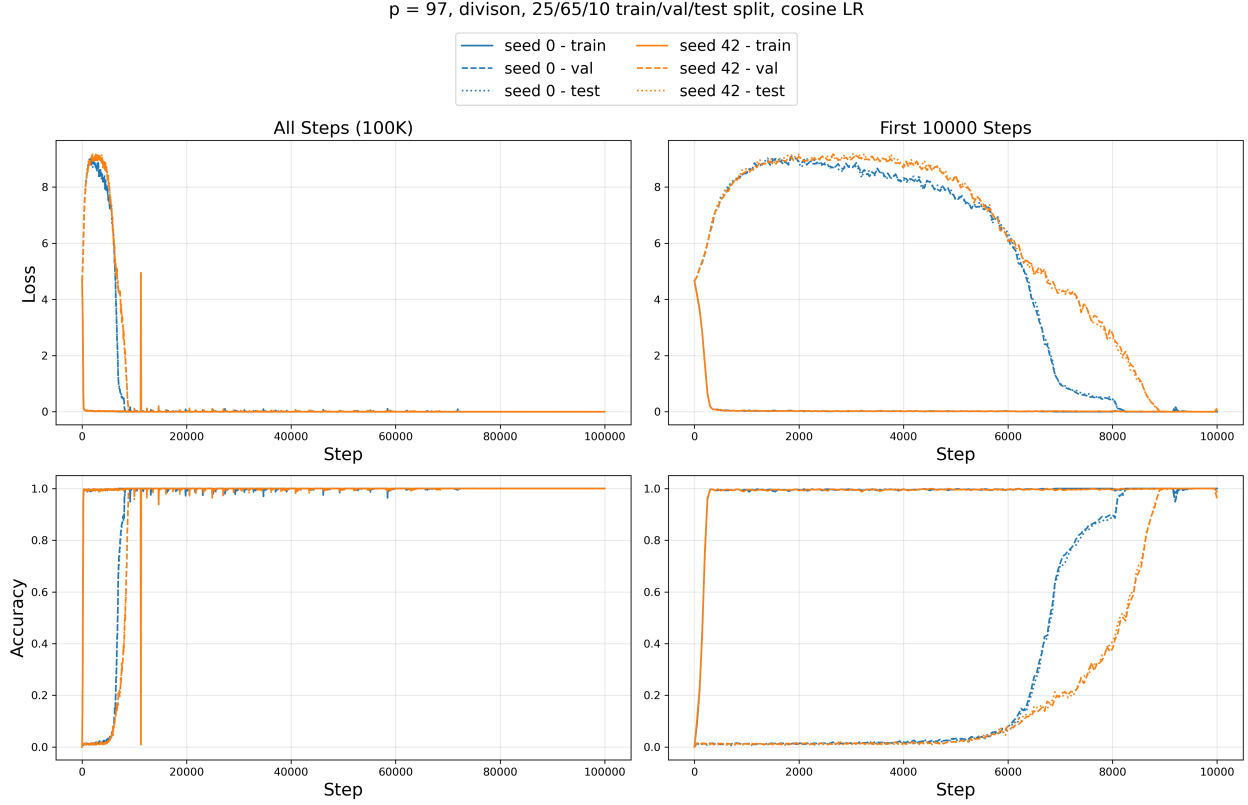


Figure 6: $p = 97$, division accuracy/loss curves, 25/65/10 train/val/test split, cosine LR

We can see that the model once again memorizes the training data within the first 1000 steps and then achieves 100% val/test accuracy around 9000 steps by grokking. Once the model groks, we can see that the val/test accuracy rarely drops far below 100%. Compared to the previous experiment, this loss curve is much smoother and would likely result in more reliable training across seeds. While the model did take longer to achieve 100% accuracy, we believe that this is due to the cosine decay learning rate. A higher learning rate would likely result in faster generalization that is comparable to previous configurations while remaining more stable.

2 Test-Time Scaling

2.1 Warm Up

We used the Qwen3-4B model to run the experiments for this section. Both Think and No Think took approximately 10 minutes each.

Exact Match Answer Parsing

	Accuracy	# Correct	# Incorrect	# Unanswered
No Think	0.267	8	15	7
Think	0.700	21	5	4

Flexible Answer Parsing

	Accuracy	# Correct	# Incorrect	# Unanswered
No Think	0.267	8	22	0
Think	0.700	21	9	0

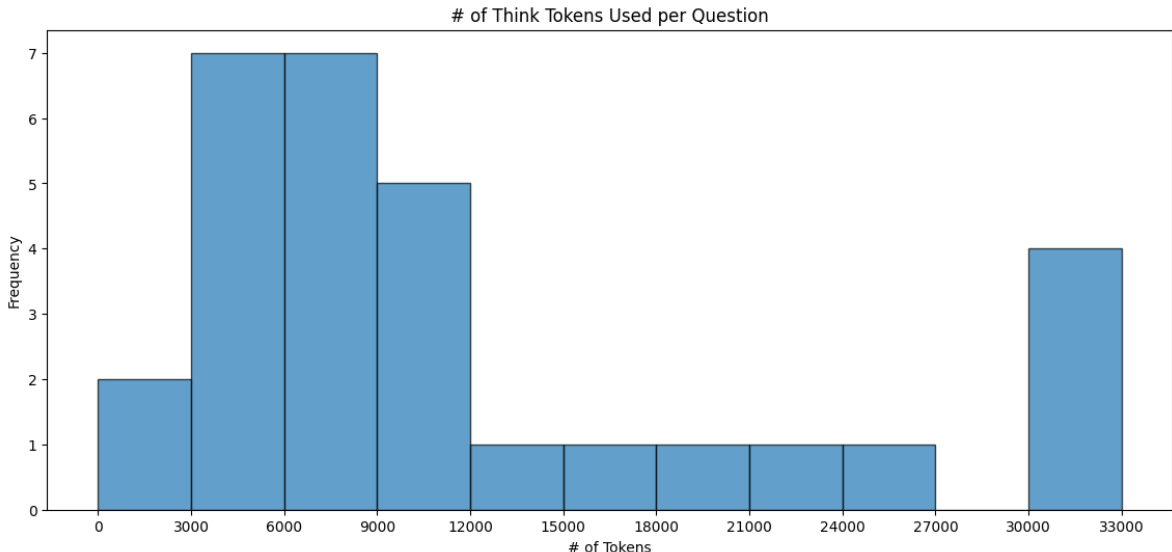


Figure 7: Histogram of the number of think tokens used per question.

2.2 Scaling Experiments

We used the Qwen3-4B model to run the experiments for this section. For the sequential scaling experiment, we enforced a think token budget on the model by:

1. Truncating the response to before the closing think token and injecting a "wait" token to encourage the LLM to continue thinking when it has not reached the budget yet
2. Truncating the response to the budget and injecting a closing think token to start the LLM on generating its answer when it hits the budget

We used budgets of 1024, 2048, 4096, 8192, 16384, and 32768 tokens for thinking, and we gave an extra 2048 tokens for the LLM to generate its answer.

For the parallel scaling experiment, we ran every sample multiple times and tracked accuracy with two metrics: the number of a times a majority of the completions agreed on the correct answer and the number of times at least one completion arrived at the correct answer. We let the model run each problem 1 time, 2 times, 4 times, 8 times, 16 times, and 32 times. We also gave the model an extra 2048 tokens to generate its answer to match the tokens given to the sequential scaling model.

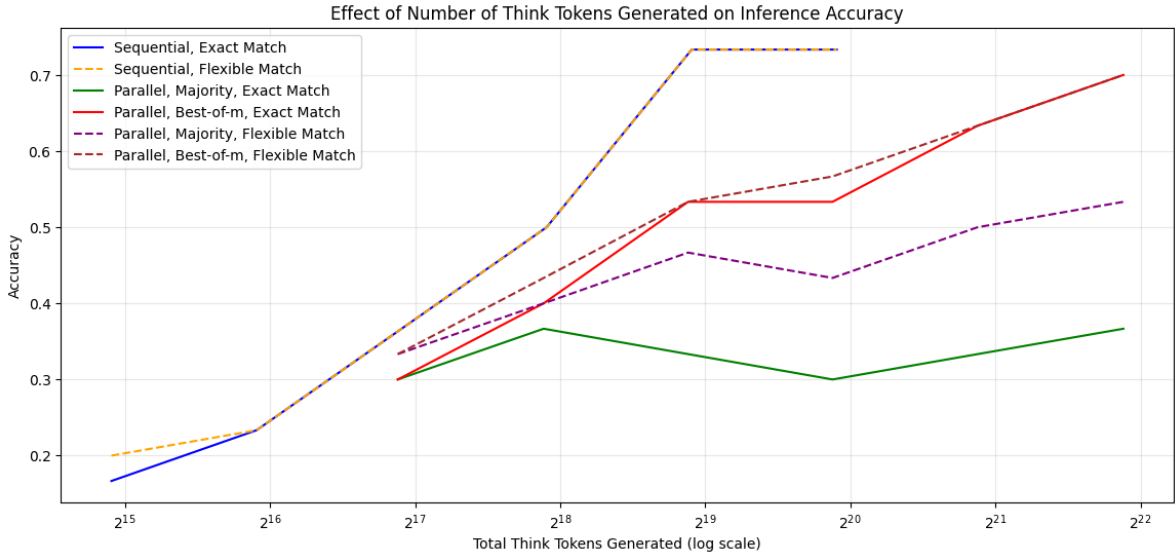


Figure 8: Relationship between accuracy and total number of think tokens used.

With total compute as a priority, sequential scaling is much better. Looking at Figure 8 and Figure 9, we can see that for any token count, sequential scaling always has a higher accuracy than any other strategy.

With latency as a priority, it is a bit more mixed. Looking at Figure 10, we can see that the parallel scaling majority voting strategy performs better than sequential scaling up to around 500 seconds, where then afterwards, sequential scaling performs better. The Best-of-M strategy consistently performs better than both of these in this metric, however, as mentioned in the homework instructions, this strategy is not a practically deployable method.

With access to an oracle to verify our answers, Best-of-M becomes a viable strategy. This changes our best strategy when optimizing for latency. As mentioned before, Best-of-M outperforms both sequential scaling and majority voting when using time spent as our metric. However, with total compute as a priority, the winner is still sequential scaling.

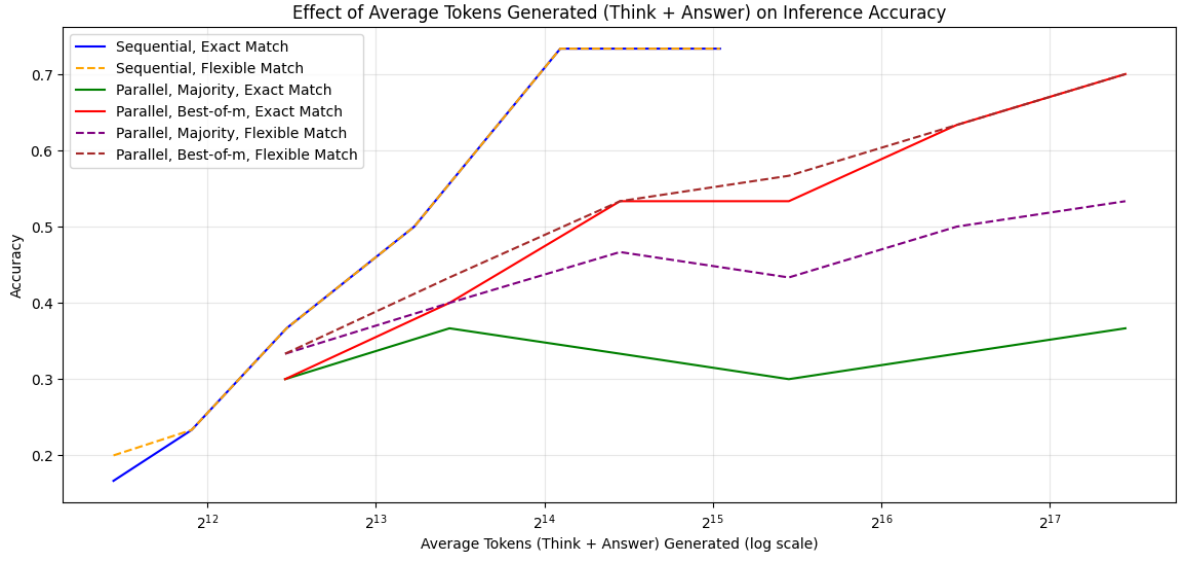


Figure 9: Relationship between accuracy and average number of tokens used per question.

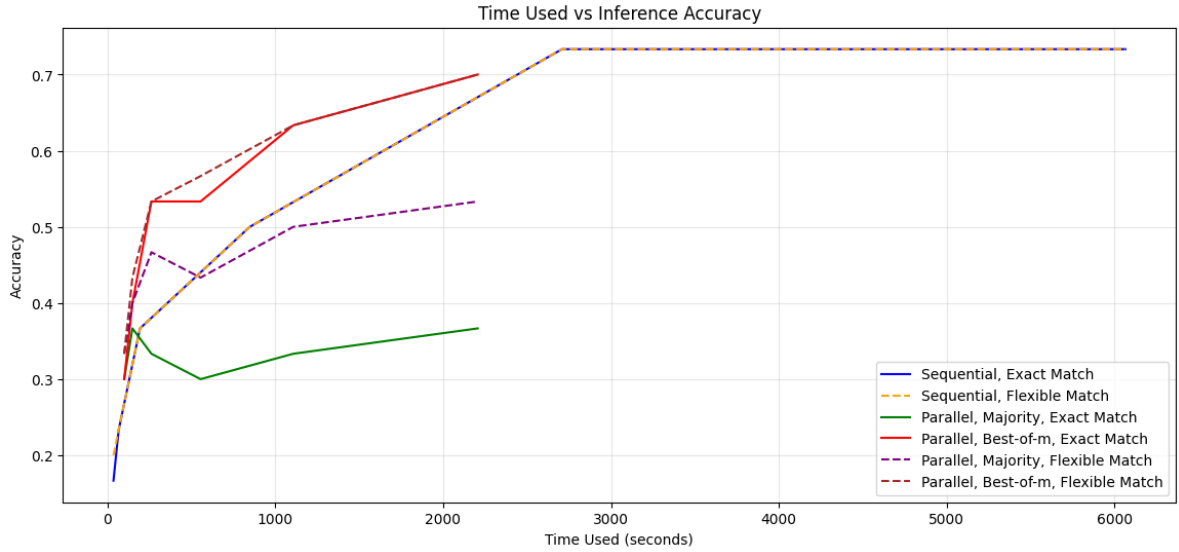


Figure 10: Relationship between accuracy and time spent.

2.3 Qualitative Analysis

Find 2 questions which are unsolved by sequential scaling at low budgets but solved at higher budgets. Do you see any change in strategy?

Question 6 remained unsolved with the 2048 token budget but was solved with the 4096 budget. Similarly, Question 8 remained unsolved with the 4096 token budget but was solved with the 8192 token budget.

Reading through the model’s train of thought, we can see that for Question 6, both models followed the same logic, starting by converting the restraints into equations and attempting to solve through algebra. However, the 2048 token model incorrectly derived that $a + b + c$ could equal either 8 or 9 when in reality, 8 was the only correct value here. Furthermore, during its explanation of the $a + b + c = 8$ case, it dropped some of its constraints midway through, leading to an incorrect answer of 800. Meanwhile, the 4096 token model correctly deduced that $a + b + c$ must equal 8, and from there, correctly figured out that this means there are only 45 different arrangements.

For Question 8, both models start off identically word-for-word, putting the two rectangles on the coordinate plane. From there, they use the equation of a circle and the quadratic equation to solve for the coordinates of E , which both models find two solutions: $(3, 16)$ and $(-187, 16)$. Both models then attempt to search for a reason to eliminate one of the answers, but only the 8192 token model noticed that $(-187, 16)$ caused the points to be in the incorrect order. From here, the 4096 token model exhausted its budget and was unable to return an answer.

Find 2 questions which are solved correctly at a lower budget but not at a higher budget for sequential scaling. What do you observe?

Question 5 was solved with the 1024 token budget. However, the 2048 token budget run was unable to solve it. Similarly, Question 19 was solved with the 8192 token budget but not with the 16384 token budget.

Reading through the model’s train of thought for Question 5, we can see that both models first calculate the result for the first few ns and notice that the losing positions occur when $n \bmod 5 \equiv 0, 2$. Then, they calculate how many losing positions there are for each case, but the 2048 budget model miscalculated and was off by one.

For Question 19, both models also follow the same reasoning. They work out that they need to use the inclusion-exclusion principle and derive the components needed to solve the equation. Furthermore, both models arrive at the correct answer of 73. However, due to the forced usage of all tokens, both models are forced to continue thinking. While the 8192 token model only needed to reconfirm 7 times to reach its budget, the 16384 token model was locked into a repeating loop where it repeated

****Final Answer****

`\boxed{73}`

Okay, I think I've confirmed this enough. The answer is 73.

for over 50% of its thinking budget. Then, after the thinking ended, the model got stuck in another loop while generating its answer, resulting in it not generating an answer. The repetition in its thinking likely influenced its repetitive answering behavior.

Are there any questions that the parallel strategy solves correctly but the sequential does not at a particular budget? Why might this be happening?

No, there does not appear to be any questions that the parallel strategy solves correctly but the sequential does not. This is likely because in the sequential strategy, every set of 4000 tokens gets to build off the previous 4000 tokens, in the parallel strategy, every set of 4000 tokens is isolated. This allows the sequential strategy to create deeper chain-of-thoughts to solve more complicated problems. Furthermore, as explained more deeply in the next question, the parallel strategy had little variation in its generation, which results in a significant amount of wasted tokens as it is essentially repeating the same statements over and over.

Do the parallel traces lead to diverse answers or solving strategies?

No, there does not appear to be much diversity in answers or solving strategies. In the 32 computations case, there was exactly one sample with 3 unique answers and all other problems had 2 or less.

There also does not appear to be much difference in solving strategy. To estimate solving strategy similarity, we first calculated an approximate embedding for each model output. To do this, we chunked the output into pieces with a length of 512 tokens, fed them through SentenceTransformer's all-MiniLM-L6-v2 model, and averaged the resulting embeddings. From this, we calculated average nearest, second nearest, and third nearest neighbor similarities, as well as the average distance to the global centroid and the standard deviation of the distance to the global centroid.

Out of all the calculated metrics, the lowest average third nearest neighbor similarity score was 0.913, and the highest average distance and standard deviation from the global centroid were 0.321 and 0.130 respectively. Furthermore, while not all responses exhibit this behavior, some PCA visualizations of the embeddings show very clear linear patterns such as the graph for Question 30.

Overall, this implies that the model did not vary much in its solving strategy throughout its repetitions.

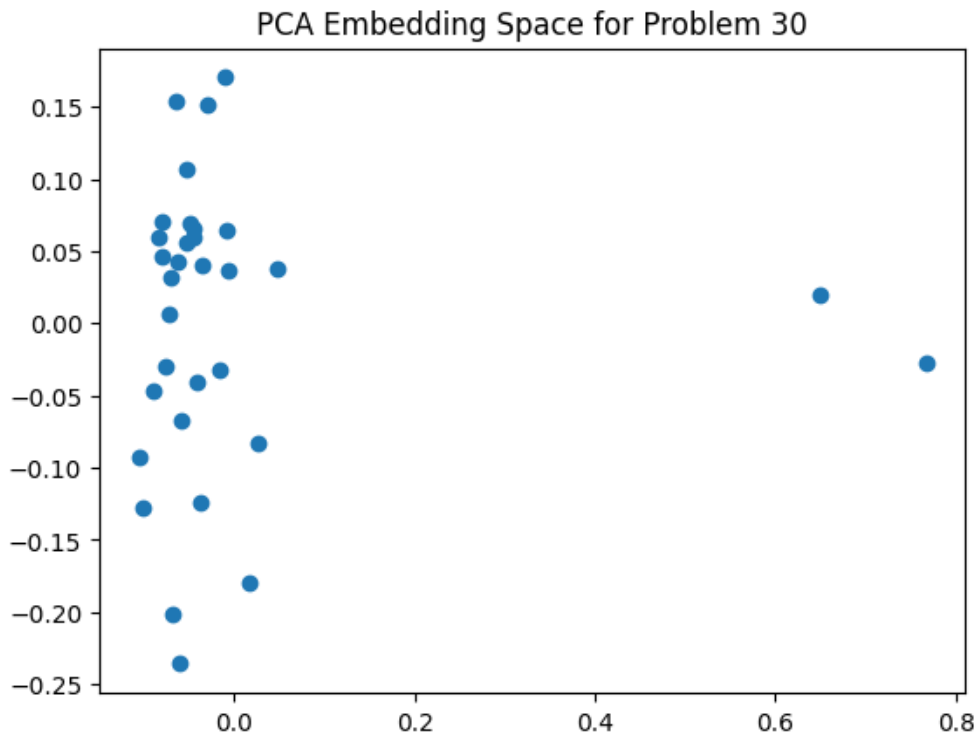


Figure 11: PCA projection of the embedding vectors of 32 responses to Question 30 into two dimensions.

2.4 Improving Parallel Scaling

Looking at all our figures from the previous part, we can see that out of the two parallel scaling tests, Best-of-M did much better than majority voting. Building on this, one of the strategies we implemented was weighing each vote with the average logit probability of its generation. We hope to see a correlation between model confidence and model correctness, which would result in a boost to the voting power of correct answers, increasing the number of correct majorities.

We also noticed a high degree of similarity among the generations for the same problem. We originally tried to remedy this by introducing XTC sampling. However, vLLM modified its API and dropped support for logit processors in its sampling parameters. We settled for modifying other sampling parameters such as increasing temperature to 0.85 to achieve similar effects. For this modification, we also restricted the strategy to a majority vote for easier comparison with the other strategy.

Weighing the votes led to a significant increase when using exact matching, with 16 correct compared to the 10 correct of the baseline for the 16,000 token budget and 13 correct compared to 9 correct for the 32,000 token budget. Flexible matching shows smaller results with 15 correct versus 14 correct on the 16,000 budget and 14 correct versus 14 correct on the 32,000 budget.

Meanwhile, increasing the temperature led to results similar to the baseline majority vote. For exact matches, this modification got 11 correct and 9 correct on the 16,000 and 32,000 budgets respectively, compared to the baselines of 10 correct and 9 correct. For flexible matches, the modification resulted in 12 correct and 14 correct compared to the 14 correct and 13 correct of the baseline on the 16,000 and 32,000 budget respectively.

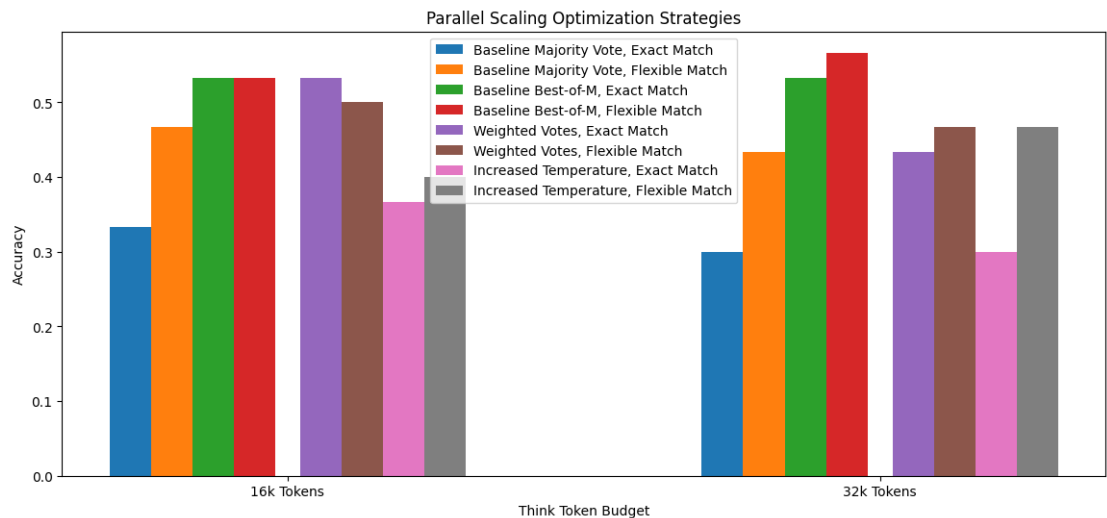


Figure 12: A bar chart comparing the accuracies of the parallel scaling modifications to the various baselines when allocated 16k and 32k thinking tokens.

3 Synthesis and Analysis

By conducting various experiments in both training and inference situations, then analyzing our results qualitatively and quantitatively, we were able to gain better insights into various approaches to model improvements. First, we found that problems that have concise rules and underlying properties benefit from extended training, as extensive training can help models learn those rules (usually, but not always). For example, with modular arithmetic, there’s rules underlying the operations (i.e., we can’t change how modular addition is performed, there’s only so many rules for the sums of different digits). We can see that this is true through our experiments in part 1, in which training with 100,000 steps allowed all of our models (all arithmetic tasks, model configurations, and seeds) to reach very close to 0 loss and 100% accuracy. Even when we used a much smaller train split of 25%, with a much larger validation/test set of 65/10, extensive training allowed our model to grok and learn the underlying rules for modular division.

In contrast, extended inference benefits more complex, multi-step problems such as those found in the AIME mathematics competition. The search space of these problems are quite large, frequently requiring one to combine several concepts across multiple domains of mathematics, such as algebra, geometry, and number theory, in order to solve. Furthermore, the correct line of reasoning is sometimes not obvious, requiring trial and error to progress. As a result, there is a substantial distance between the problem and the answer, requiring reasoning through intermediate steps and sometimes even backtracking to explore other avenues. Extended inference simulates this process, giving a model more space to look deeper into multiple lines-of-reasoning. Our experiments support this. Looking at the sequential scaling experiment, when the model only received 1024 tokens for reasoning, it achieved a 17% accuracy. However, when that limit was increased to 32 thousand tokens, the model’s accuracy had increased to 73%. Between these two datapoints, accuracy consistently increased as it was allocated more tokens, suggesting that these additional tokens allowed it to better navigate these problems.

This relates to a commonality between grokking failing to occur under certain configurations, longer thinking budgets hurting accuracy, and parallel samples lacking diversity. When we attempt to take advantage of our high compute resources without strategy, we may cause our models to fail to generalize. Data is often limited; we often don’t have access to all possible inputs. When we try to leverage compute to learn our data better, our models may learn patterns and dynamics in the data that do not actually exist. For example, if there is artifacts in the data that do not relate to the output, increasing compute to learn the data may cause our inferences to use those artifacts.

Too much compute during inference time can also lead to adverse results. As mentioned in our scaling experiments, sometimes the model with more compute failed a question that was solved by a model with less compute. As mentioned before, one of our models inferreded itself into a loop due to being forced to use all its tokens while it was reasoning. Even after being kicked out of the loop from running out of think tokens, the damage was already done and this generation history caused the model to fall into another generation loop while

answering, resulting in no final answer outputted. As a result, compute and generalization essentially have an inverse relationship. Especially in extreme cases, leveraging too much compute will reducing our model’s accuracy and efficiency. Thus, it is important to maintain a balance between leveraging compute and generalization techniques to not only maintain our models’ accuracies, but also robustness.

Using the insights that we found through our experiments, a strategy for designing a model for solving a range of math problem difficulties (easy, medium, hard) while given a fixed compute budget and a single pretrained reasoning model, is the following. For easy problems, we will prioritize our compute budget for training. We believe that this would be the most efficient approach, as easy problems will often have a direct answer that does not require detailed reasoning or verification ³. For example, modular arithmetic is an easy math task, in which we found that prioritizing training easily helped us achieve the task by allowing our model to learn the underlying rules and dynamics. For medium problems, we would balance our compute budget for both training and inference. Medium problems will introduce dynamics and properties that may require more adaption, learning, and reasoning, which inference-related approaches will greatly help. We know this from the part 2 experiments, where using 1024 think tokens + 2048 answer tokens achieved 17% accuracy, while 32768 think tokens + 2048 answer tokens achieved 73% accuracy. Lastly, for harder problems, we will shift our focus to mainly inference. Harder problems often require in-depth reasoning, solution exploration, and result verification, which are all capabilities that are primarily influenced by increased inference-time compute ^{4 5}. While a finetuned understanding of the problem field can improve performance when solving difficult problems, this can be achieved by using a small portion of the compute budget for training. Finetuning has diminishing returns as training compute scales, so even a small amount of finetuning on a subject matter can provide effective performance gains.

³<https://arxiv.org/html/2510.03605v1>

⁴<https://arxiv.org/html/2502.12521v1>

⁵<https://neurips.cc/virtual/2025/loc/san-diego/poster/121550>